

Github Action - Play - Run

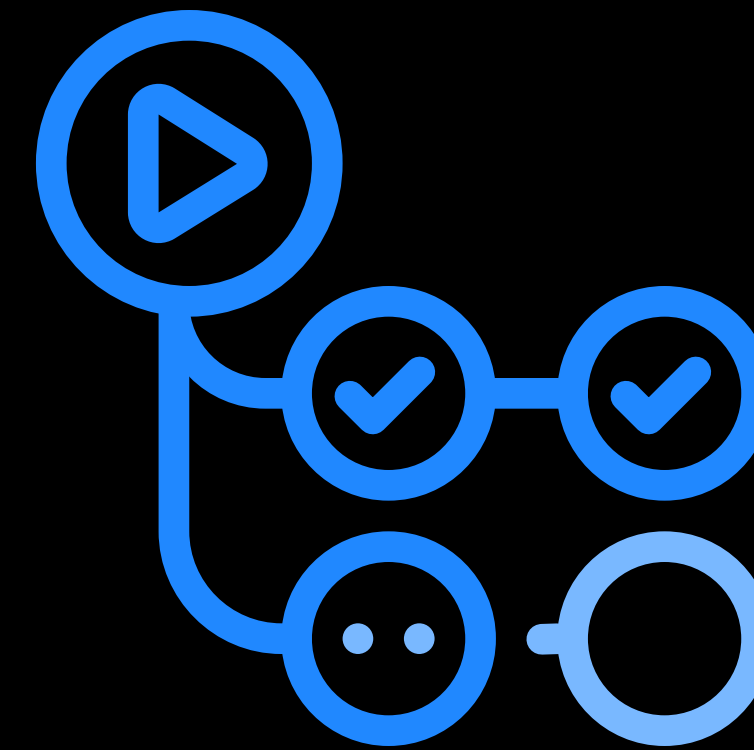
A fundamental walkthrough of **GitHub Action**

Keshav Mohta | February 2026

Agenda

A brief what we will learn together

- What are GitHub Actions ?
- Components of GitHub Actions
- Workflow syntax
- Variables and Secrets
- Composite Actions
- Reusable Workflows
- Demo
- QnA



What are GitHub Actions ?

GitHub Actions makes it easy to automate all your software workflows, now with world-class CI/CD.

*GitHub Actions goes beyond just DevOps and lets you run **workflows** when other **events** happen in your repository, like issue, PR, commits, discussions and many more.*

Components of GitHub Actions

Anatomy of actions

- **Workflows** ( `.github/workflows/<what-ever>.yml`) – process
- **Events** (denoted with `on`) – activity having sub events and activity type
- **Jobs** (pre-built action / shell script / `matrix` strategy) – set of steps, runs in parallel / series
- **Actions** ( `.github/actions/<which-ever>/action.yml`) – pre-defined/
custom, reusable set of jobs
- **Runners** (denoted with `runs-on`) – server

Workflow Syntax

```
name: GitHub Actions Demo → Name of Workflow
run-name: ${{ github.actor }} is testing out GitHub Actions 🚀
on: [push] → Event
jobs: → Jobs
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest → Runner
    steps:
      - run: echo "🎉 The job was automatically triggered by a ${{ github.event_name }} event."
      - run: echo "🐙 This job is now running on a ${{ runner.os }} server hosted by GitHub!"
      - run:
echo "🔍 The name of your branch is ${{ github.ref }} and your repository is ${{ github.repository }}."
      - name: Check out repository code
        uses: actions/checkout@v5 → Actions
      - run: echo "💡 The ${{ github.repository }} repository has been cloned to the runner."
      - run: echo "🖨 The workflow is now ready to test your code on the runner."
      - name: List files in the repository
        run: |
          ls ${{ github.workspace }}
      - run: echo "🍏 This job's status is ${{ job.status }}."
```

Workflow Syntax

Events / Sub Events / Category Type (supports 40 +)

Variation 1 → inline list ✓

```
~~~
on: [push, pull_request, 'workflow_dispatch']
branches: [main, develop, '!test/**']
types: [opened, labeled]
~~~
```

Variation 2 → block list ✓

```
~~~
on:
  branches:
    - main
    - 'feature/**'
    - '!release/**'
  paths:
    - 'docs/**'
~~~
```

Variation 3 → inline item ✓

```
~~~
on: push
branches-ignore: main
tag-ignore: 'v1.**'
~~~
```

Variation 4 → nested list ✗ (YAML ✓)

```
~~~
on:
  branches:
    - main
    - [develop, release]
~~~
```


Workflow Syntax

Jobs

```
jobs:
  my-first-job: # unique job id; space not allowed
    name: My First Job # visible under GitHub workflow UI
    environment: staging # create environment under the repo from GitHub UI and write that name
    runs-on: ubuntu-latest

    permissions: # can set direct value read-all | write-all
      issues: write
      pull-requests: read

    steps:
      - id: checkout-repo
        name: my first step - checkout repo
        if: ${ github.ref_name == 'main' } # can omit ${ }
        uses: actions/checkout@v5 # checkout to repository

      - run: npm install
        shell: bash # default for ubuntu runner

      - uses: actions/setup-node@v4 # setup node
        with:
          node-version: "20" # write variable which defined by this action

      - run: | # multiline command using |
          npm run lint
          npm run test --if-present
```

`uses` --> “import a tool”
`run` --> “write the instructions yourself”
⚠ `uses` and `run` never combined in one step.

Variables and Secrets

Private, encrypted and masked

- Create environment from GitHub UI and store secrets and variables << **Recommended**
- Repo variables accessed using `${{ vars. }}`
- Repo secrets accessed using `${{ secrets. }}`
- variables defined under `env` accessed using `${{ env. }}`
- github reference metadata can be accessed using `${{ github. }}` // readonly
- Runtime variables can be accessed using `${{ inputs. }}` ;
- variable passed to composite action / reusable workflow via `with`

GitHub Variables

Predefined variables and secrets

- `GITHUB_TOKEN` is special variable name.
- Whenever use github cli ``gh``
 - in workflow

```
env:  
  GH_TOKEN: {{ secrets.GITHUB_TOKEN }}
```
 - in composite action

```
env:  
  GH_TOKEN: {{ github.token }}
```
- `GITHUB_TOKEN` is empirical i.e. auto generated during runtime.
- `steps:`
 - name: some task
 - id: `build`
 - run: `echo "version=1.2.3" >> $GITHUB_OUTPUT`

can be accessed in other steps using `steps.build.outputs.version`
- Other default files
 - `$GITHUB_ENV`
 - `$GITHUB_PATH`
 - `$GITHUB_STEP_SUMMARY`
 - `$GITHUB_STATE`

Workflow syntax

env: custom variable defined and used in the workflow

Workflow level

```
~~~
name: Format Issue Title
env:
  PROJECT_NAME: "Action Club"
~~~
- available everywhere in this workflow
```

Access syntax

- inside `run` → `$VAR`
e.g. - `run: echo "Project is $PROJECT_NAME"`
- inside `'env'`, `'with'` `'if'` → `${{ env.VAR }}`
e.g.
 - `if: env.PROJECT_NAME == 'Action Club'`
 - `env:`
 - `GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}`
 - `FULL_NAME: "${{ env.PROJECT_NAME }}-build"`

Step level

```
~~~
steps:
  - name: Add Prefix
    env:
      PROJECT_PREFIX: "ACL"
~~~
```

Demo

Composite Actions

Composite actions contain a series of steps that are run as a single step within the caller workflow.

- actions file Inside **.github/actions** folder name as **action.yml**
- **action.yml** run must contains **using: "composite"** and **steps**
- accept parameters under **inputs** ; and pass using **with** from caller workflow
- run as a **step** within a **jobs** in caller workflow
 - **uses: {owner}/{repo}/.github/actions/{folder-name}@{ref}** (from another repo)
 - or
 - **uses: ../.github/action/action-name** (from same repo)

{ref} can be a SHA, a release tag, or a branch name.
- composite action does not contain **jobs**
- We can run javascript action using **actions/github-script@v8** and under **script** key

Reusable Workflows

A workflow to avoid duplication

- A workflow that uses another workflow is referred to as a "*caller*" workflow. The reusable workflow is a "*called*" workflow. One caller workflow can use multiple called workflows.
- *reusable* workflow must have
 - on:
 - workflow_call:
- define *input* and *secrets* keyword using in *called* workflow
- pass value using *with* from *caller* workflow

Trivia

- One can publish your action freely to the GitHub Marketplace (one action per repo)
- use *spaces* not *tabs* while writing YAML.
- `::` is workflow command protocol ; `echo " :: add-mask :: my-secret-value "`
- wiki is separate repo comes free with the repo (project name *gollum*)

`https://github.com/<OWNER>/<REPO>.wiki.git`
- Maximum workflow runtime: 35 days
- Maximum job/step run time: 6 hrs
- Max upload limit: 5GB per file; 50GB total per run
- Max Secrets: 1000
- Max logs retention: 90days
- GitHub bot id: `41898282` // `github-actions[bot]` whenever use `GITHUB_TOKEN`
 - GitHub generate a no reply email for each user in format (check ID on `https://api.github.com/users/<username>`)

`<userId>+<username>@users.noreply.github.com`

QnA

Thanks for your time.

- <https://xkeshav.com>
- `const social<any> = @xkeshav`
- run `npx xkeshav` to know more

Keep learning => Keep sharing :)

